JMOA - Build-Time JVM Memory Optimization for Spring Boot

A portfolio of JVM memory optimization case studies showing that bytecode rewriting only becomes a product result when packaging, runtime policy, origin verification, and container memory measurement are aligned.

Problem

Spring Boot memory density is affected by lambdas, adapter classes, class metadata, packaging mode, classloader visibility, CDS/no-CDS mode, allocator behavior, and launch shape. The relevant question is whether fewer classes become lower PSS and Private_Dirty in the measured deployment.

Approach

JMOA profiles workload behavior, selects candidate lambda/adaptor sites, rewrites bytecode at build time, consolidates PACKAGE_SAM adapters, materializes a runtime-valid artifact, verifies runtime class origins, then measures PSS, Private_Dirty, cgroup memory.current, NMT, and class histograms.

Confirmed Results

Service	Runtime mode	CDS?	Confirmed result
Patient-service	expanded classpath	yes	~4.2-4.4 MB median memory reduction
Doctor-service	corrected Spring Boot fat JAR	yes	~2.7 MB median PSS reduction
Spring PetClinic customers-service	exploded Boot / JarLauncher	no	~4.6 MB median PSS reduction

Public Centerpiece

Spring PetClinic customers-service confirmed a public no-CDS result: full P2, exploded Boot / JarLauncher, no CDS/AppCDS/Leyden, no runtime javaagent, MALLOC_ARENA_MAX=1, dynamic runtime origins verified, and 3/3 paired wins. Median deltas: PSS -4.6 MB, Private_Dirty -4.8 MB, memory.current -4.6 MB, heap PSS -7.0 MB, loaded classes -154.

Key Lesson

JMOA is not only a bytecode optimizer. It is a JVM deployment optimizer: candidate selection + bytecode correctness + adapter placement + artifact materialization + deployment-shape alignment + runtime policy + runtime-origin verification + paired measurement.

Claim Integrity

Do not claim JMOA always wins. Do not cite Doctor Phase 32I or the mistaken -5.9 MB median. Do not mix Patient 31D-P2 with 31E. Do not claim PetClinic wins under fat-JAR mode or that MALLOC_ARENA_MAX=1 alone solved no-CDS memory.